

The Convolution, Mathematically

optional deep-dive appendix

L16 gave the working intuition – slide a kernel, take a dot product – and stated the convolution theorem as a short aside.

This appendix embeds LMU's *full* treatment for anyone who wants the mathematics underneath: the flip-and-slide definition, the convolution theorem *with proof*, and the formal convolution-vs-cross-correlation story.

What this appendix adds beyond L16

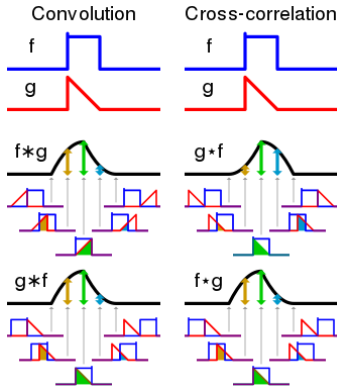
L16 already gave you everything you need to *use* convolutions. Read this only if you want the math under them. The genuinely new content here:

- ▶ **The flip.** True convolution *flips* the kernel before sliding. The 1D animation (next) shows the flip-and-slide that L16's footnote only named.
- ▶ **The convolution theorem, proved.** L16 stated $\mathcal{F}\{f * g\} = \mathcal{F}\{f\} \mathcal{F}\{g\}$; here is the derivation (a change of variables under the integral – Fubini).
- ▶ **Cross-correlation, formally.** Its own definition and animation, then the side-by-side with convolution – and why `nn.Conv2d` computes cross-correlation and nobody minds.
- ▶ **Two bonus threads** (from 3Blue1Brown, at the very end): what a convolution looks like in **2D** - a kernel blurring an image - and the FFT trick that computes a convolution in $O(n \log n)$.

The two long stretches that follow are LMU's, verbatim: click-through **animations** (1D convolution, then 1D cross-correlation) – advance them with the arrow key. A short house-authored bonus closes the deck.

Deep Learning

Convolutions - Mathematical Perspective



Learning goals

- Convolution vs. Cross-Correlation

CONVOLUTIONS : A DEEPER LOOK

- CNNs borrow their name from a mathematical operation termed **convolution** that originates in Signal Processing.
- Basic understanding of this concept and related operations improves the understanding of the CNN functionality.

CONVOLUTIONS : A DEEPER LOOK

- Definition:

$$h(i) = (f * g)(i) = \int f(x)g(i - x)dx$$

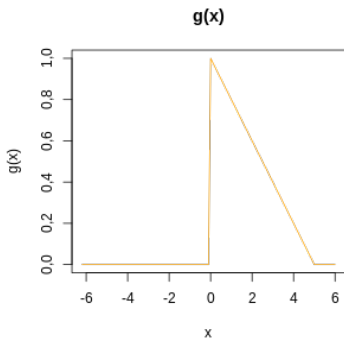
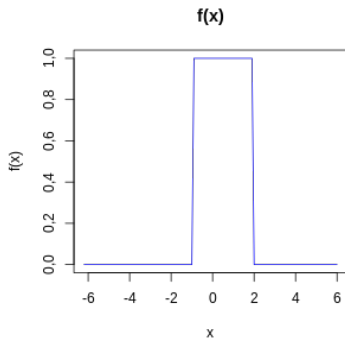
where $f(x)$: input function

and $g(x)$: weighting function, kernel

and $h(i)$: output function, feature map elements

- Intuition 1: weighted smoothing of $f(x)$ with weighting function $g(x)$.
- Intuition 2: filter function $g(x)$ filters features $h(i)$ from input signal $f(x)$.

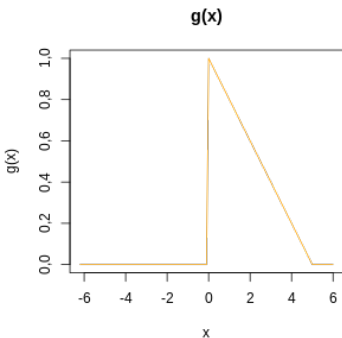
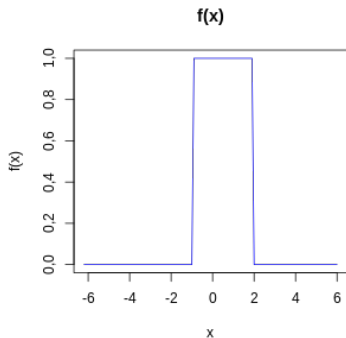
1D CONVOLUTION ANIMATION



$$f(x) = \begin{cases} 1, & \text{if } x \in [-1, 2] \\ 0, & \text{otherwise} \end{cases}$$

$$g(x) = \begin{cases} 1 - 0.2 * |x|, & \text{if } x > 0 \\ 0, & \text{otherwise} \end{cases}$$

1D CONVOLUTION ANIMATION

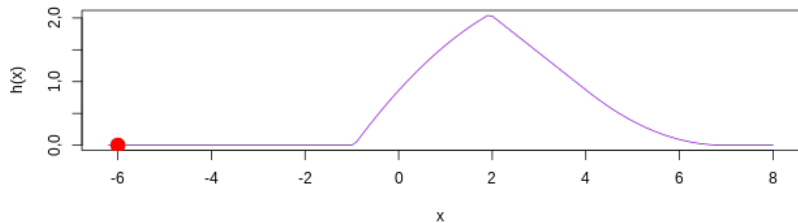
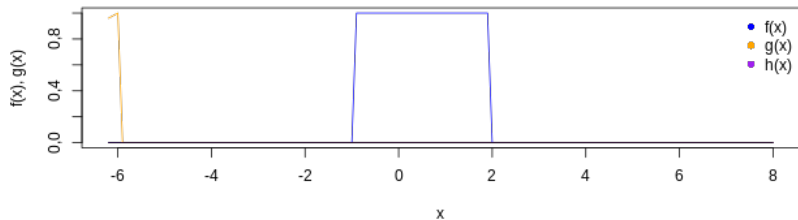


Kernel is flipped due to the negative iterator in

$$h(i) = \int_{x=-\infty}^{\infty} f(x)g(i-x)$$

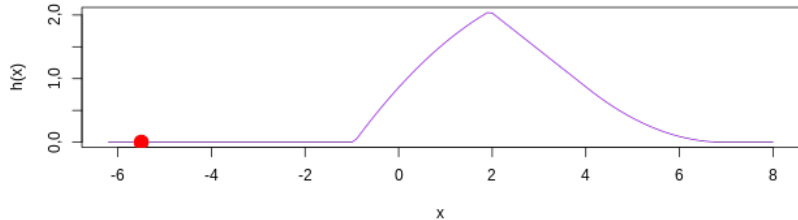
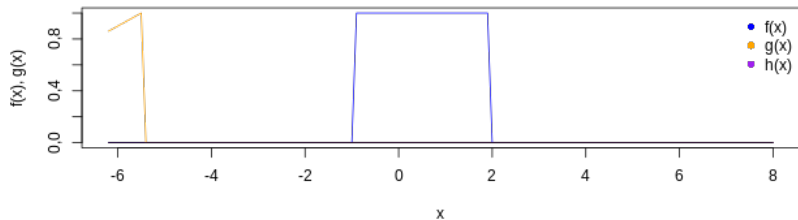
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



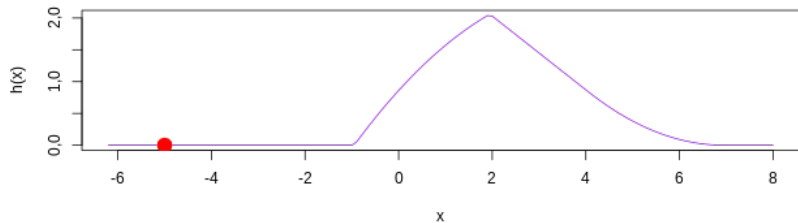
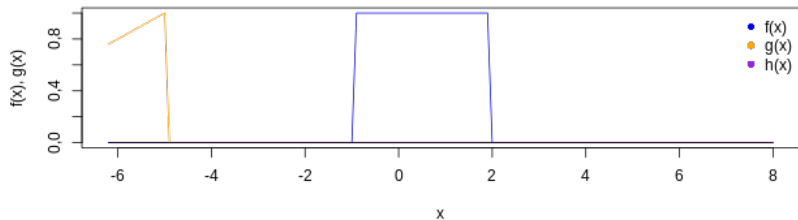
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



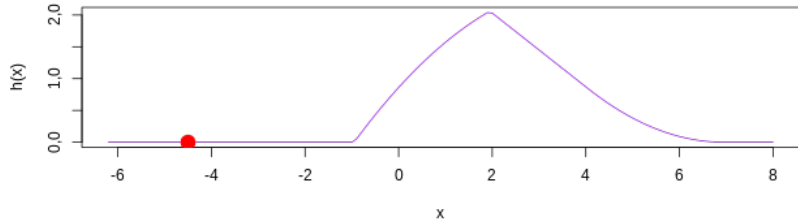
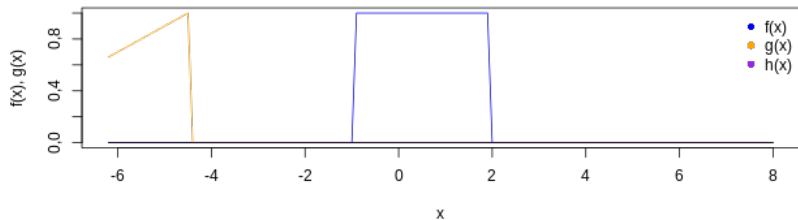
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



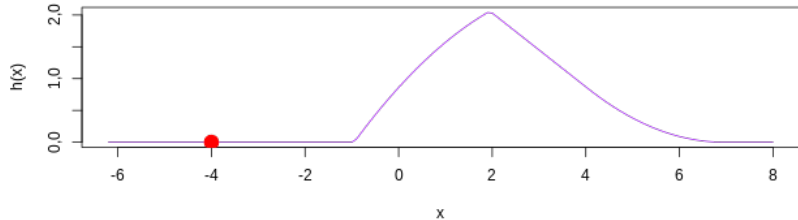
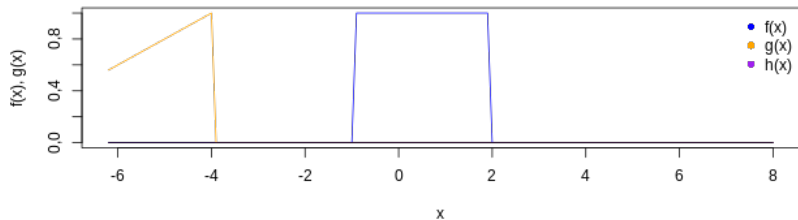
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



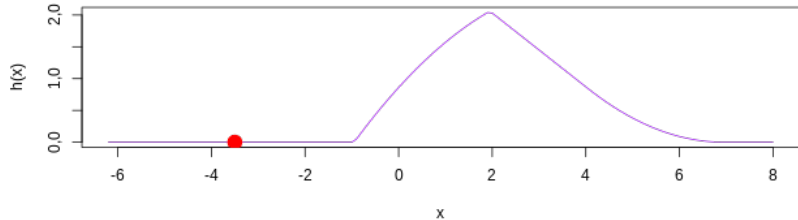
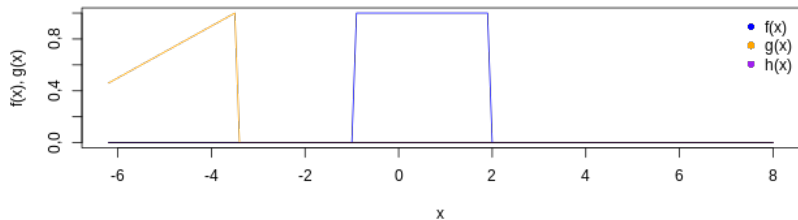
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



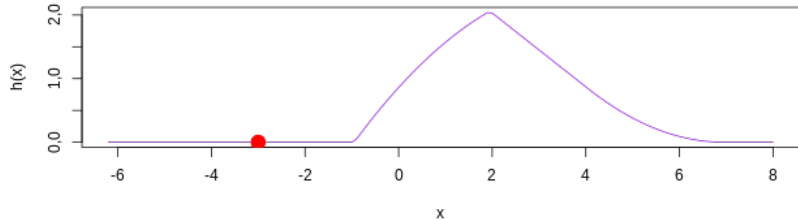
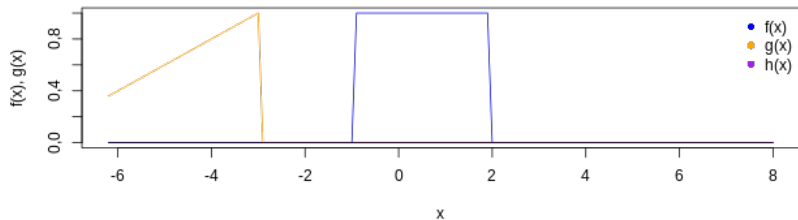
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



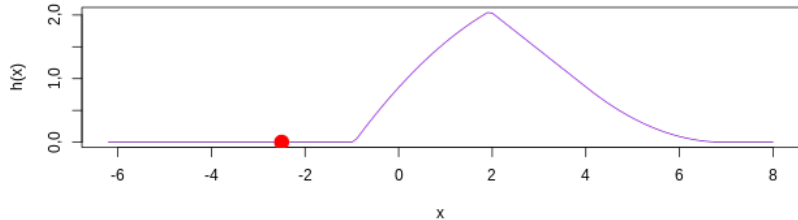
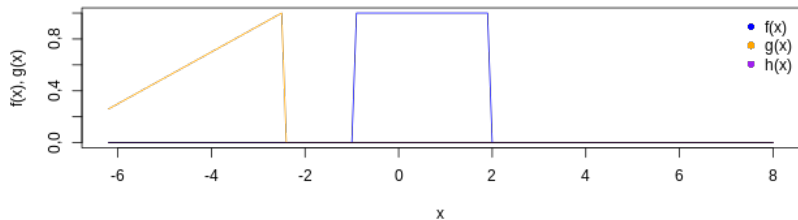
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



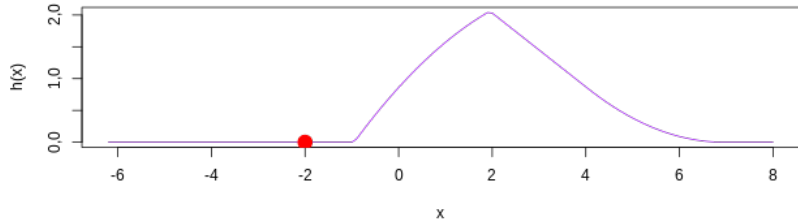
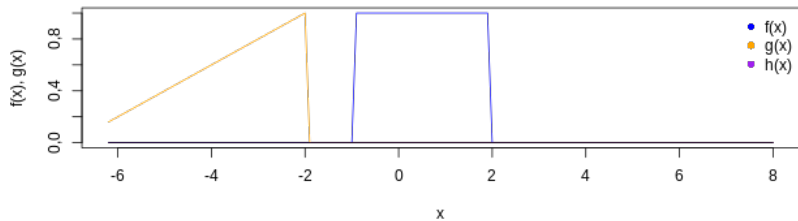
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



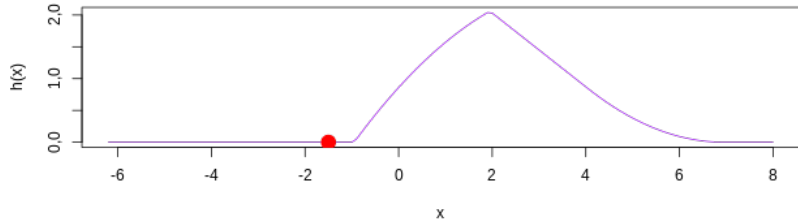
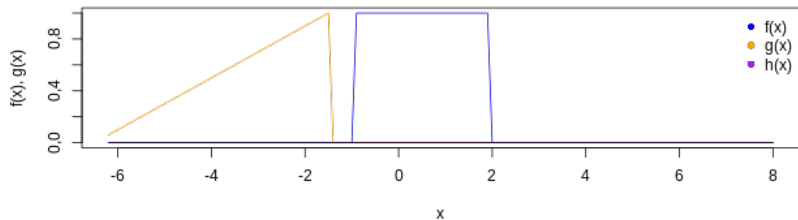
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



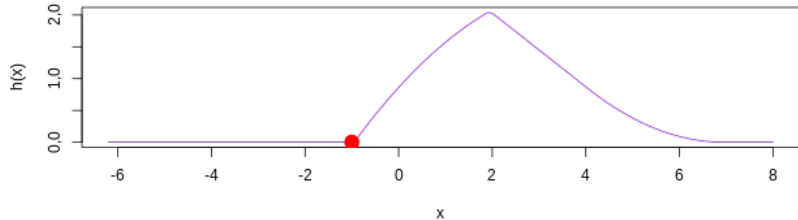
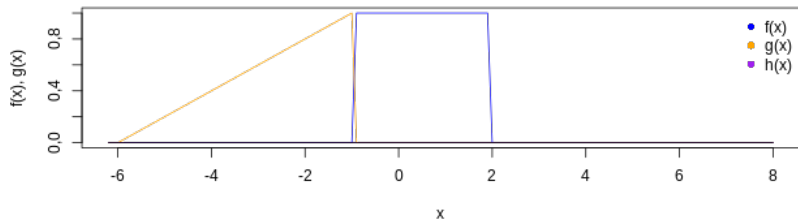
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



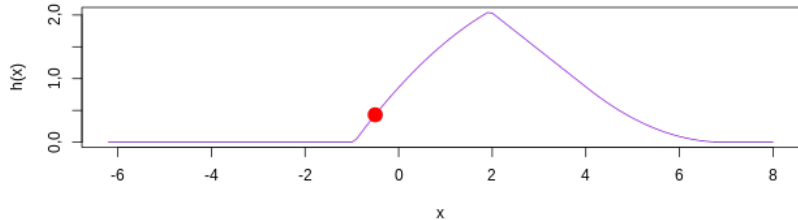
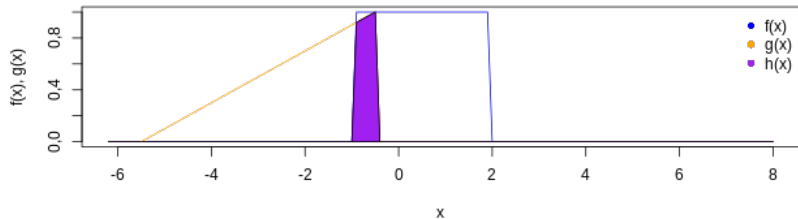
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



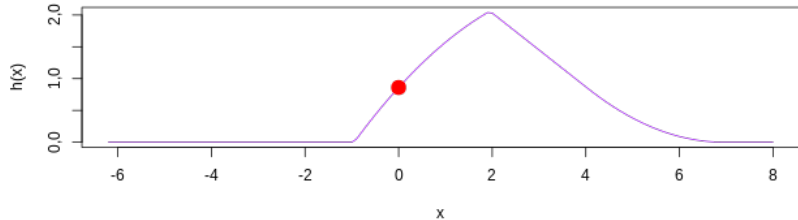
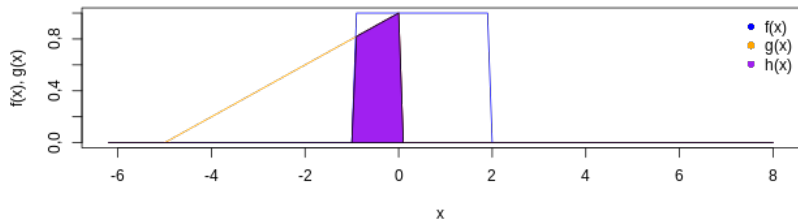
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



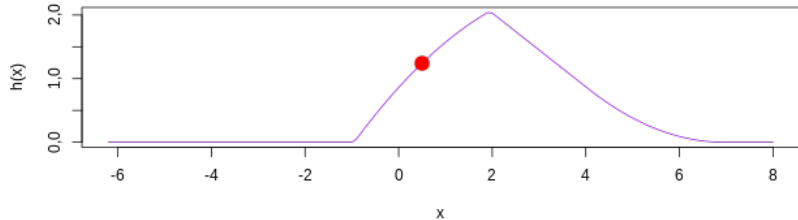
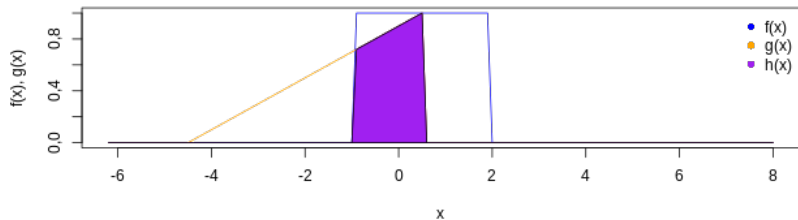
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



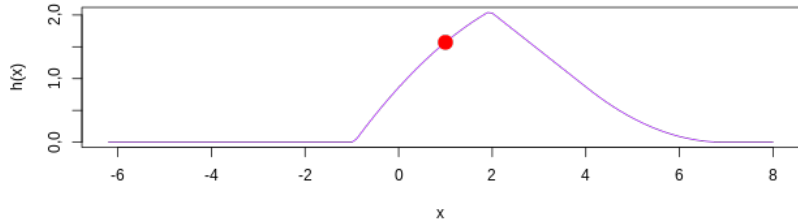
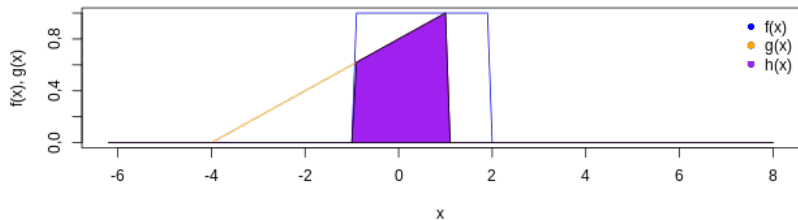
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



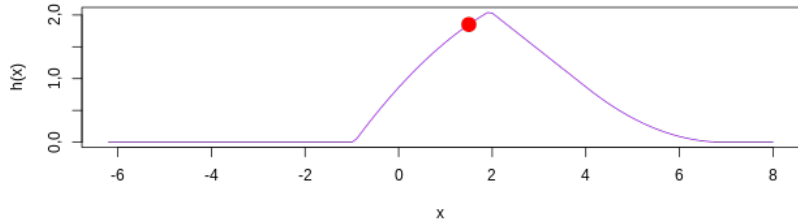
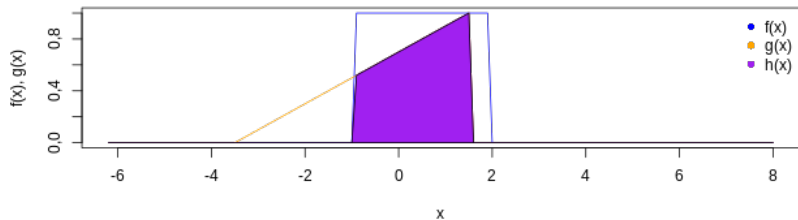
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



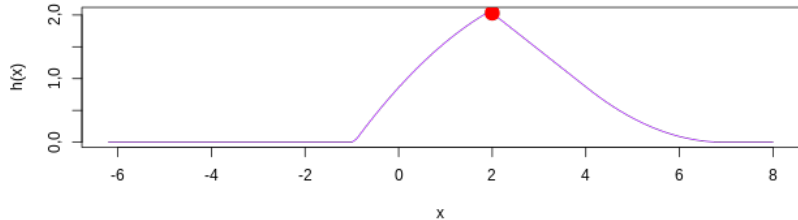
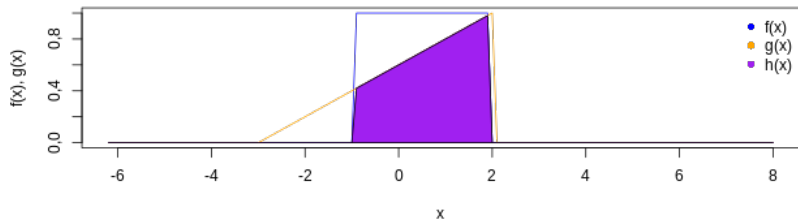
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



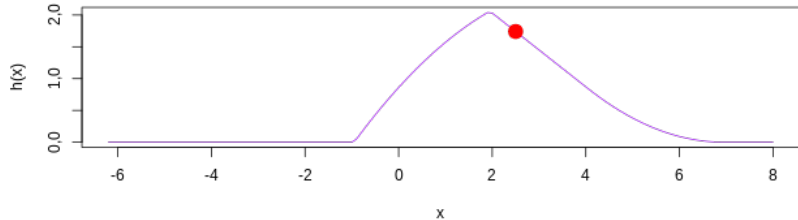
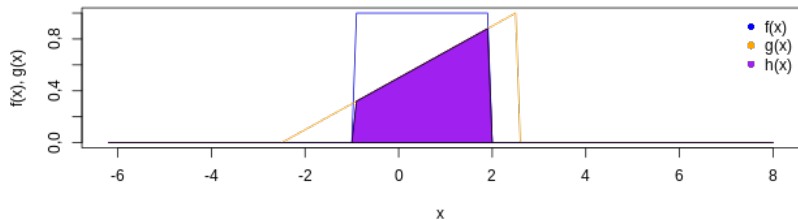
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



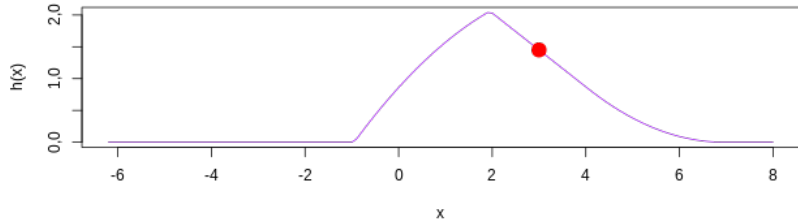
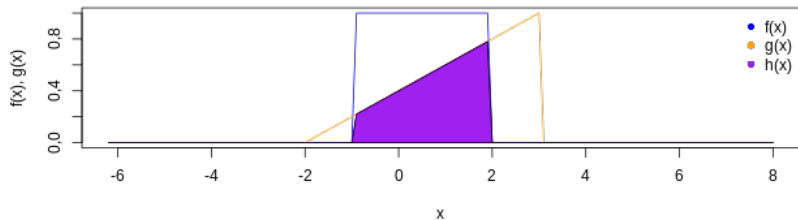
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



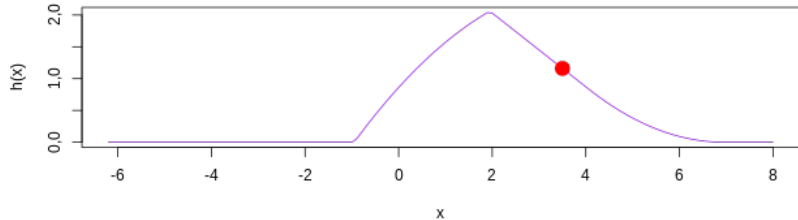
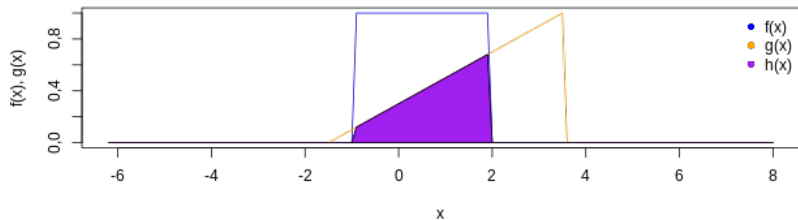
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



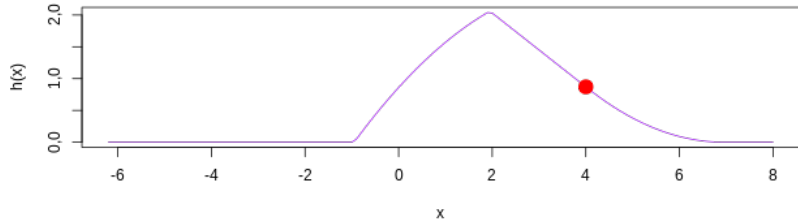
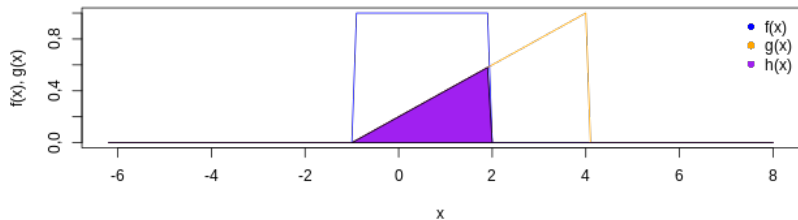
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



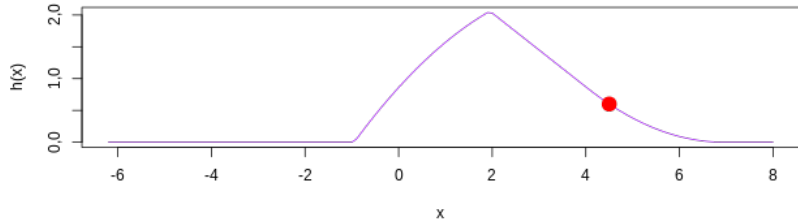
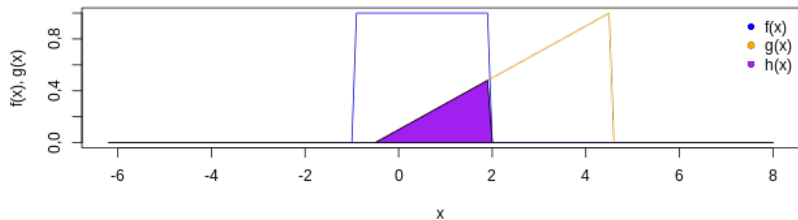
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



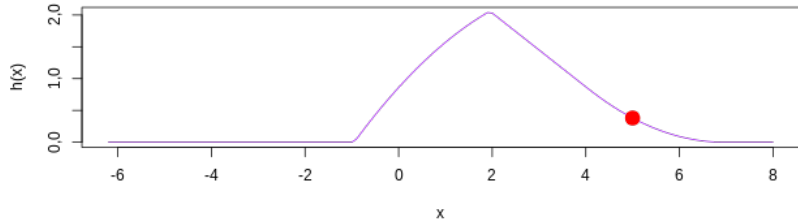
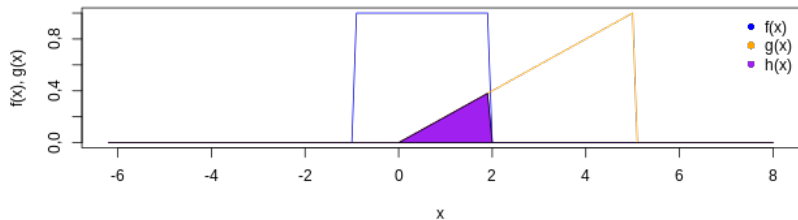
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



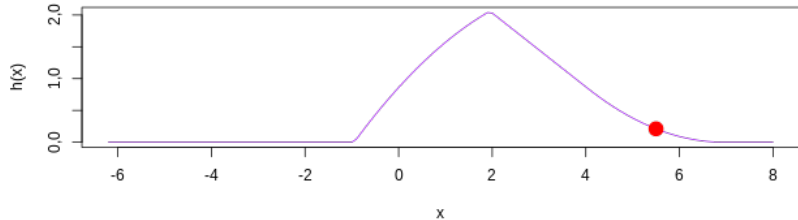
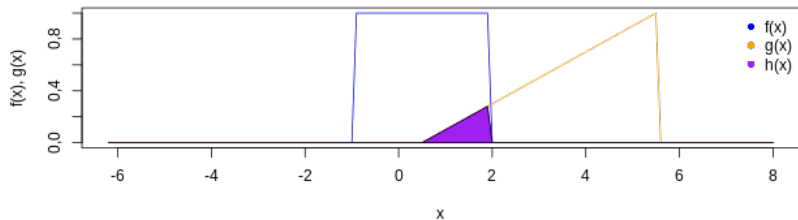
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



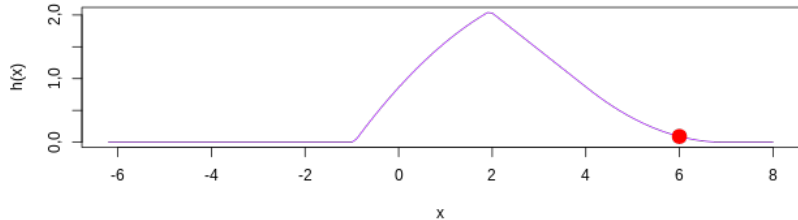
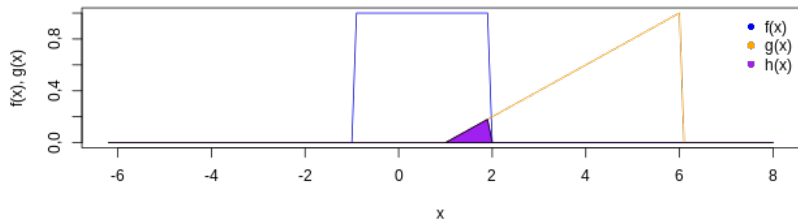
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



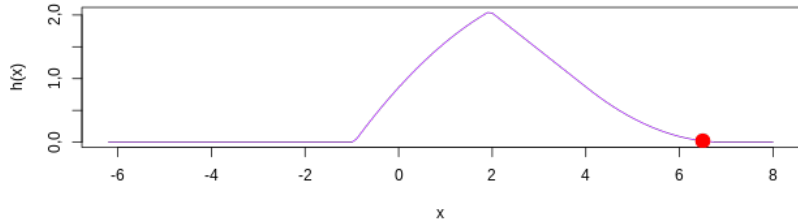
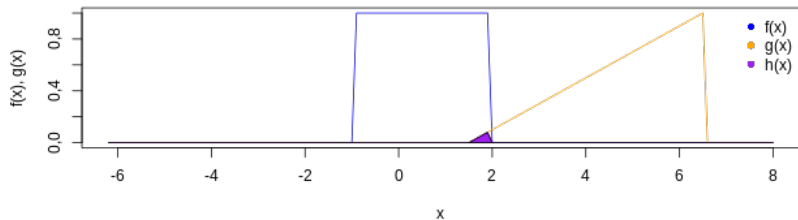
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



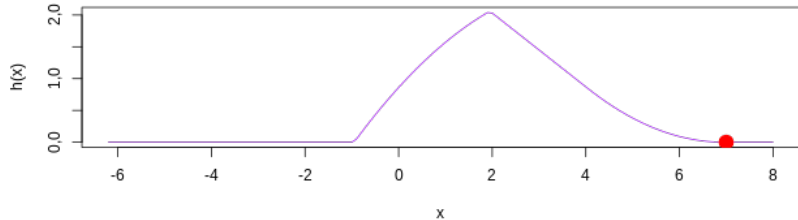
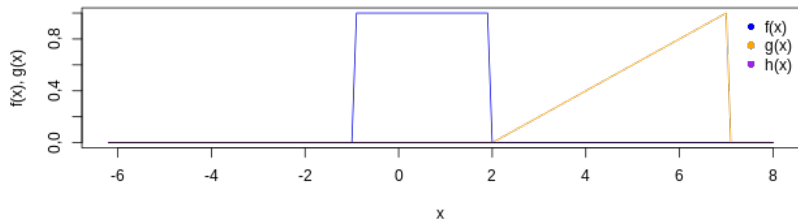
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



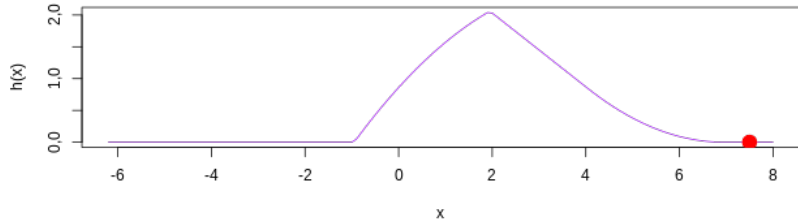
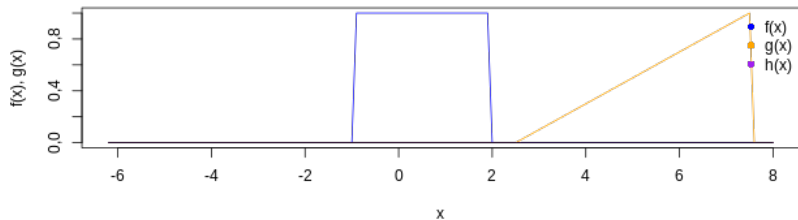
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



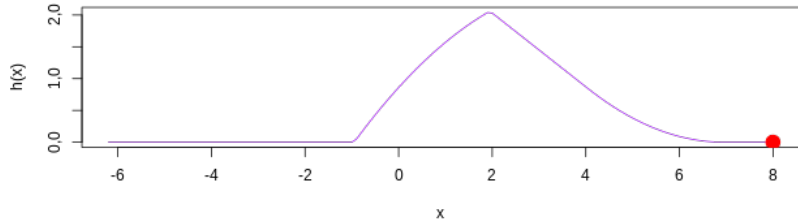
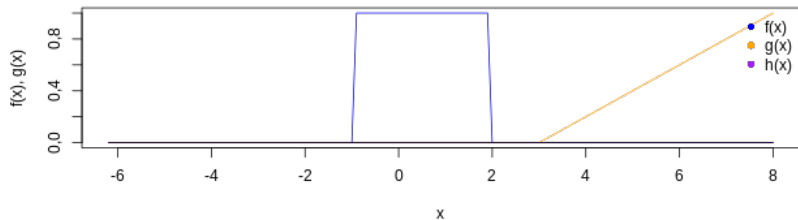
1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



1D CONVOLUTION ANIMATION

Convolution of $f(x)$ with $g(x)$



DISCRETIZATION

- Discretization for one-dimensional input:

$$h(i) = (f * g)(i) = \sum_x f(x)g(i - x)$$

- Discretization for 2D images:
 - $\mathcal{I} \in \mathcal{R}^2$ contains two dimensions
 - Use 2D Kernel $\mathcal{G}$ as well to yield feature map $\mathcal{H}$:

$$H(i, j) = (\mathcal{I} * \mathcal{G})(i, j) = \sum_x \sum_y \mathcal{I}(x, y) \mathcal{G}(i - x, j - y)$$

where $x, y :=$ indices $\mathcal{I}$ and $\mathcal{G}$

and $i, j :=$ indices elements in $\mathcal{H}$

PROPERTIES OF THE CONVOLUTION

- Commutativity:

$$f * g = g * f$$

- Associativity:

$$(f * g) * h = f * (g * h)$$

- Distributivity:

$$f * (g + h) = f * g + f * h$$

$$\alpha(f * g) = (\alpha f) * g \text{ for scalar } \alpha$$

- Differentiability:

$$\frac{\partial(f * g)(x)}{\partial x_i} = \frac{\partial f(x)}{\partial x_i} * g(x) = \frac{\partial g(x)}{\partial x_i} * f(x)$$

$\rightarrow (f * g)(x)$ is as many times differentiable as the max of $g(x)$ and $f(x)$.

RELATED OPERATIONS

- Convolution is strongly related to two other mathematical operators:
 - ① Fourier transform via the Convolution Theorem
 - ② Cross correlation

CONVOLUTION THEOREM

- Fourier transform of the convolution of two functions can be expressed as the product of their Fourier transforms:

$$\mathcal{F}\{f * g\} = \mathcal{F}\{f\}\mathcal{F}\{g\}$$

- Transformation of a signal from time to frequency domain.
- Convolution in the time domain is equivalent to multiplication in frequency domain.
- The computationally fastest way to compute a convolution is therefore to take the Fourier inverse of the multiplication of the Fourier-transformed input and filter function :

$$(f * g)(t) = \mathcal{F}^{-1}\{\mathcal{F}\{f(t)\}\mathcal{F}\{g(t)\}\}$$

CONVOLUTION THEOREM - PROOF

$$\begin{aligned}\widehat{(f * g)}(t) &= \int_{-\infty}^{\infty} \exp(-2\pi i \omega t) \left[\int_{-\infty}^{\infty} f(\tau) g(t - \tau) d\tau \right] \\&= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \exp(-2\pi i \omega t) f(\tau) g(t - \tau) d\tau dt \\&\stackrel{\text{Fubini}}{=} \int_{-\infty}^{\infty} \left[\int_{-\infty}^{\infty} \exp(-2\pi i \omega t) f(\tau) g(t - \tau) dt \right] d\tau \\&\stackrel{f(\tau) \perp t}{=} \int_{-\infty}^{\infty} f(\tau) \left[\int_{-\infty}^{\infty} \exp(-2\pi i \omega t) g(t - \tau) dt \right] d\tau \\&\stackrel{u=t-\tau}{=} \int_{-\infty}^{\infty} f(\tau) \left[\int_{-\infty}^{\infty} \exp(-2\pi i \omega \tau) \exp(-2\pi i \omega u) g(u) du \right] d\tau \\&= \int_{-\infty}^{\infty} \exp(-2\pi i \omega \tau) f(\tau) \left[\int_{-\infty}^{\infty} \exp(-2\pi i \omega u) g(u) du \right] d\tau \\&\stackrel{\text{Fubini}}{=} \dots\end{aligned}$$

CONVOLUTION THEOREM - PROOF

$$\begin{aligned} & \dots \int_{-\infty}^{\infty} \exp(-2\pi i \omega \tau) f(\tau) d\tau \int_{-\infty}^{\infty} \exp(-2\pi i \omega u) g(u) du \\ & = \hat{f}(t) \hat{g}(t) \end{aligned}$$

From convolution to cross-correlation

You have just seen *true* convolution: flip the kernel, then slide. What every deep-learning framework actually computes drops the flip – that is **cross-correlation**.

- ▶ Same slide-and-sum, *without* reversing the kernel.
- ▶ This is L16's footnote made precise: `nn.Conv2d` is cross-correlation wearing convolution's name.
- ▶ It changes nothing in practice – the kernel is **learned**, so the network would just learn the flipped weights either way.

The next stretch is LMU's formal cross-correlation: definition, its own click-through animation, and the final side-by-side comparison.

CROSS CORRELATION

- Measurement for similarity of two functions $f(x)$, $g(x)$.
- More specifically, at which position are the two functions most similar to each other? Where does the pattern of $g(x)$ match $f(x)$ the best?
- Intuition:
 - Slide with $g(x)$ over $f(x)$ and at each discrete step compute the sum of the product of their elements.
 - When peaks of both functions are aligned, the product of high (positive or negative) values will lead to high sums.
 - Thus, both functions are most similar at points with equal peaks.

CROSS CORRELATION

- Definition:

$$h(i) = (f \star g)(i) = \int_{-\infty}^{\infty} f(x)g(i+x)dx$$

where $f(x)$: input function

and $g(x)$: weighting function, kernel

and $h(i)$: output function, feature map elements

for $f, g \in \mathcal{R}^d \mapsto h \in \mathcal{R}^d$

CROSS CORRELATION

- Discrete formulation:

$$h(i) = (f \star g)(i) = \sum_{x=-\infty}^{\infty} f(x)g(i+x)$$

- Thus:

$$f(i) \star g(i) = f(-i) * g(i)$$

- Remember: $*$ is used for convolution and $\star$ for cross correlation.
- Similar formulation as the convolution despite the flipped filter function in the convolutional kernel .

CROSS CORRELATION

- This operation also works in 2 dimensions
- The difference w.r.t. the convolution are the positive iterators in the sum:

$$H(i, j) = (\mathcal{I} \star \mathcal{G})(i, j) = \sum_x \sum_y \mathcal{I}(x, y) \mathcal{G}(i + x, j + y)$$

where $x, y :=$ indices $\mathcal{I}$ and $\mathcal{G}$

and $i, j :=$ indices elements in $\mathcal{H}$

CROSS CORRELATION

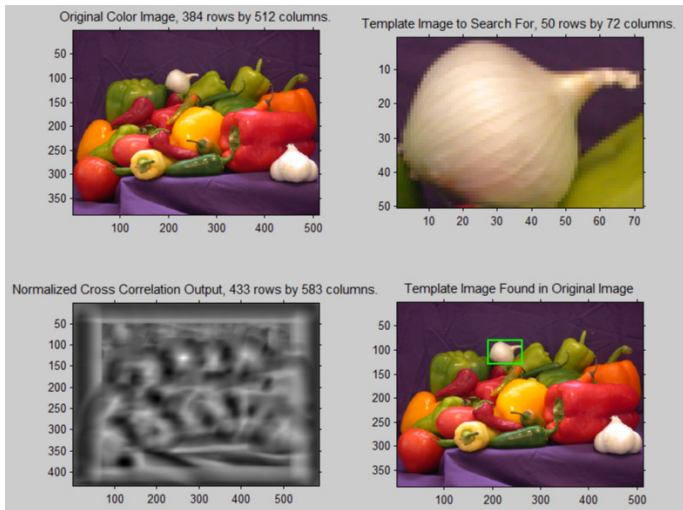


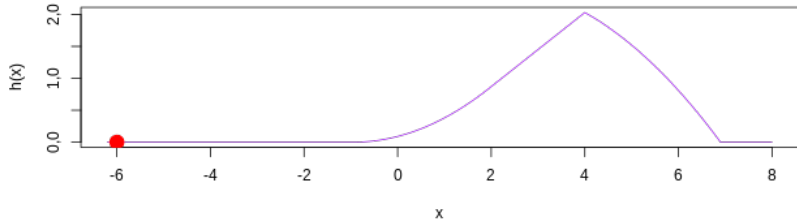
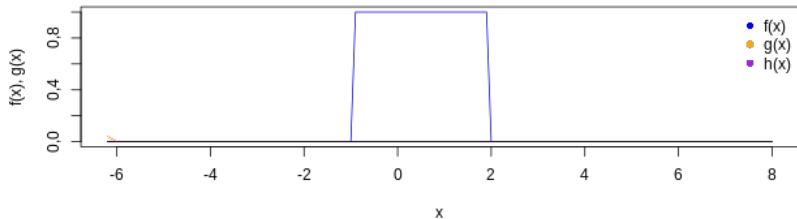
Figure: Cross-correlation used to detect a template (onion) in an image. Cross correlation peaks (white) at the position where template and input match best.

CROSS CORRELATION

- From the following animation we see that
 - Kernel is not flipped as opposed to the convolution.
 - Cross-Correlation peaks, where the filter matches the signal the most.
- In some frameworks, Cross-Correlation is implemented instead of the convolution due to
 - better computational performance.
 - similar properties, as the kernel weights are learned throughout the training process.

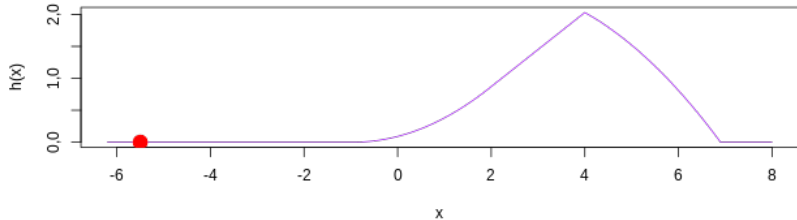
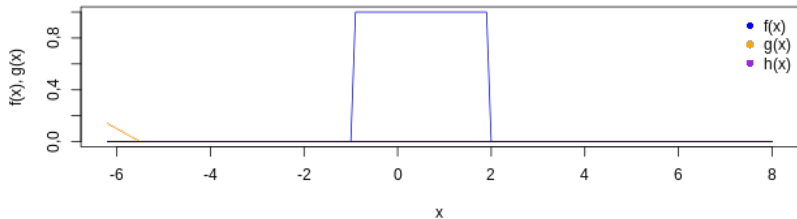
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



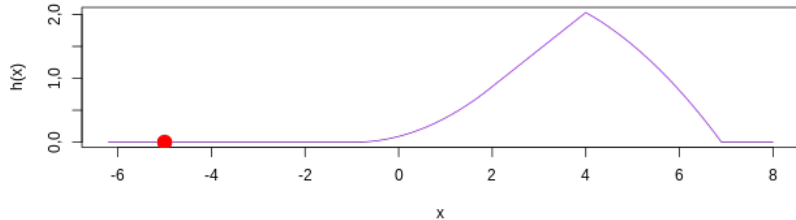
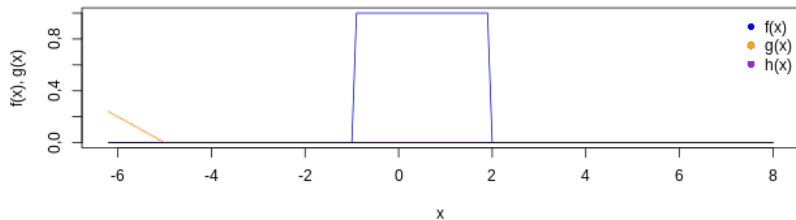
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



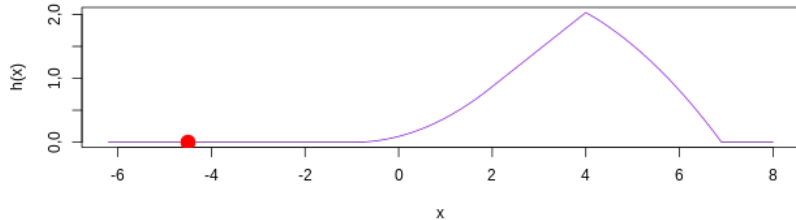
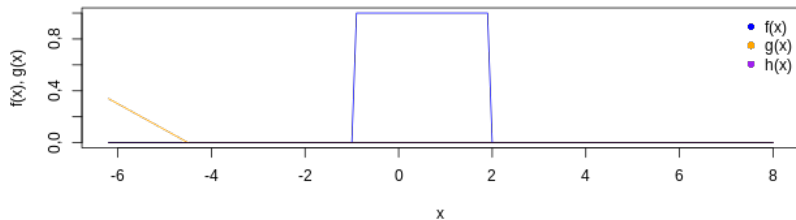
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



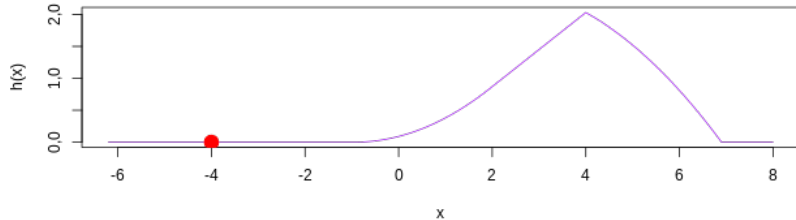
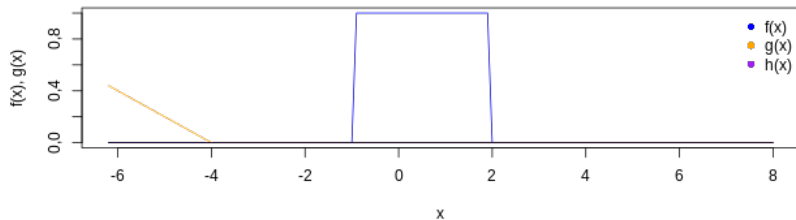
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



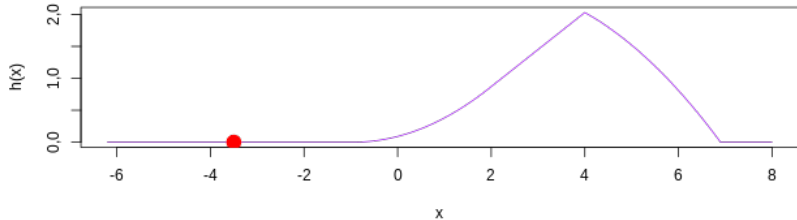
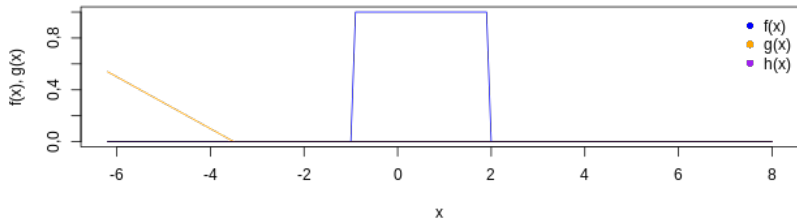
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



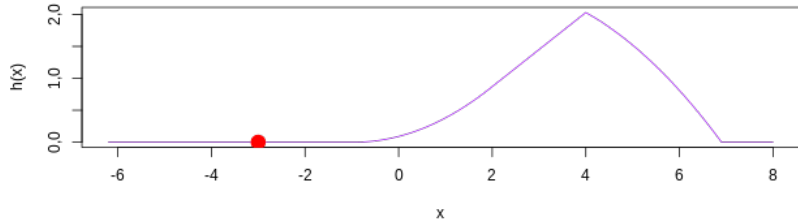
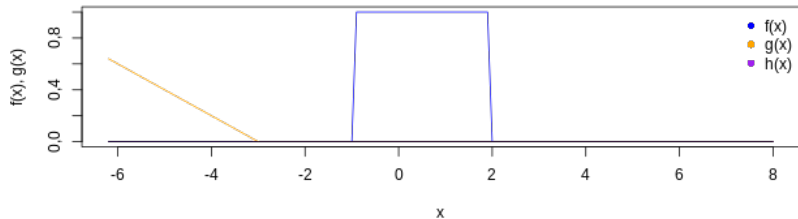
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



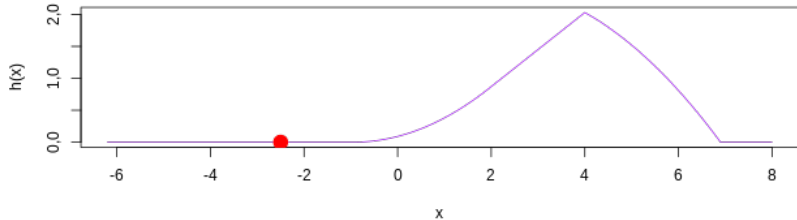
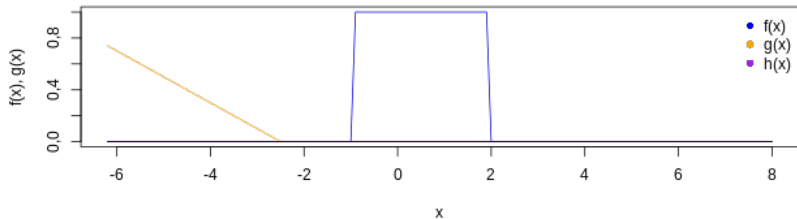
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



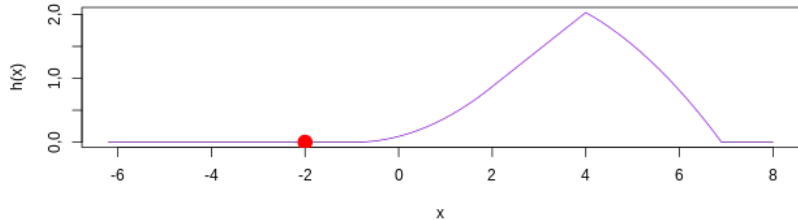
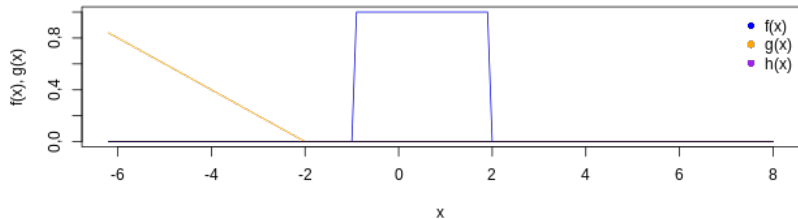
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



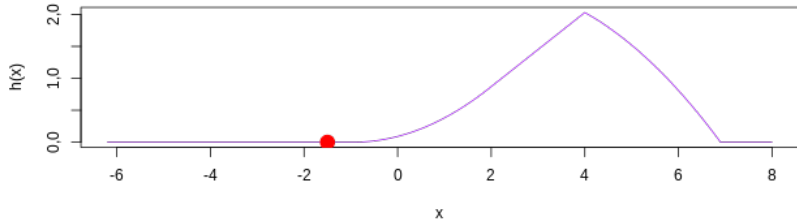
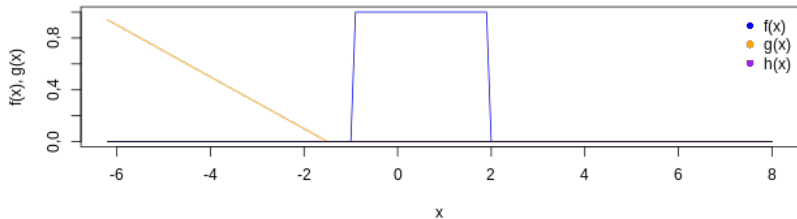
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



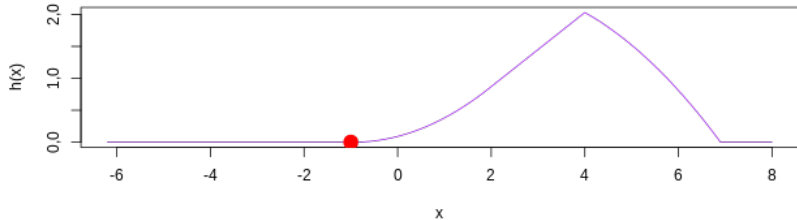
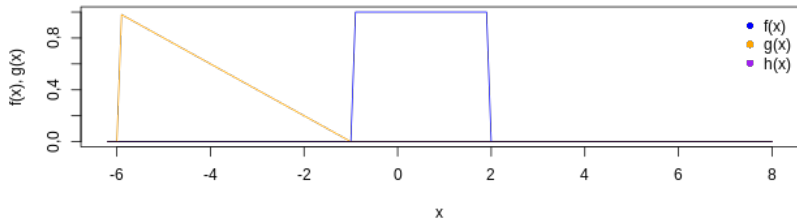
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



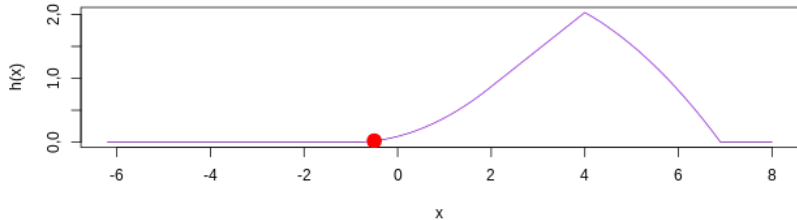
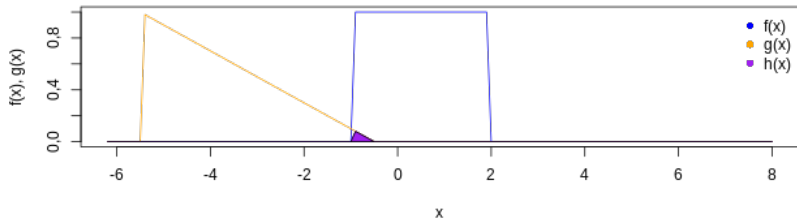
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



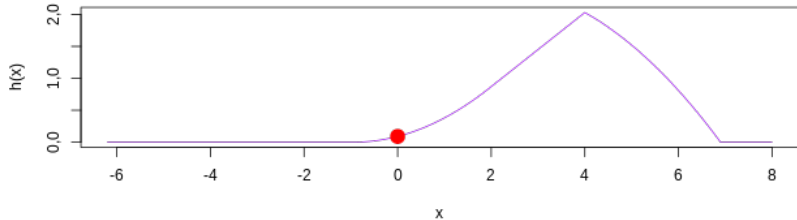
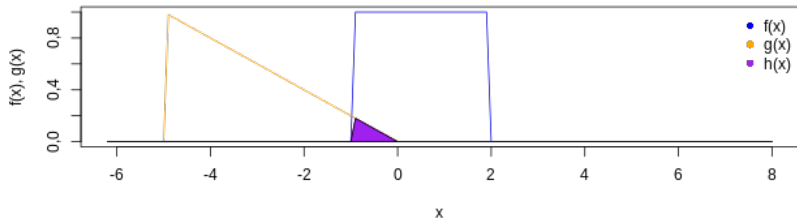
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



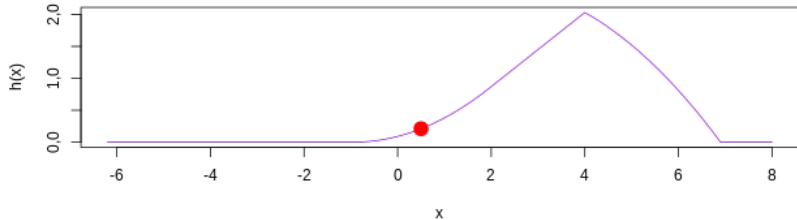
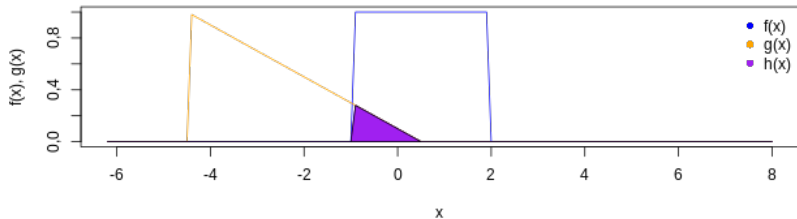
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



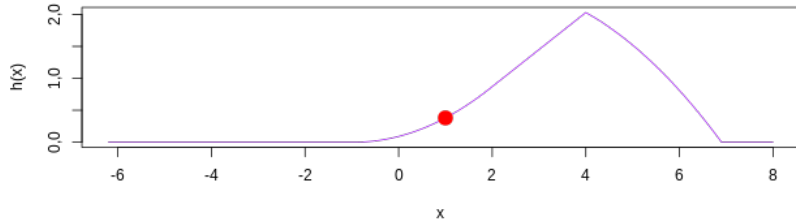
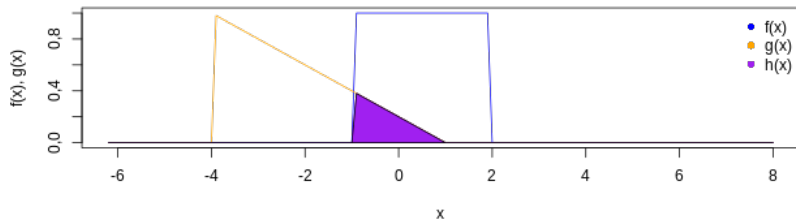
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



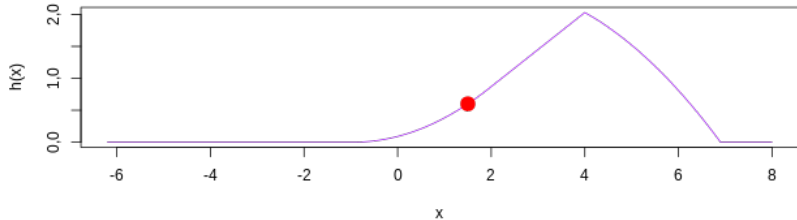
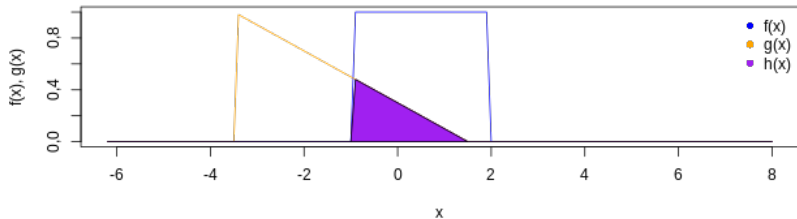
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



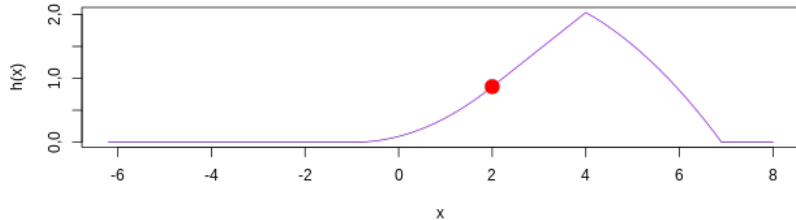
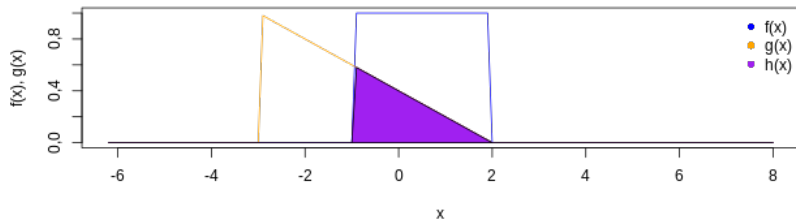
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



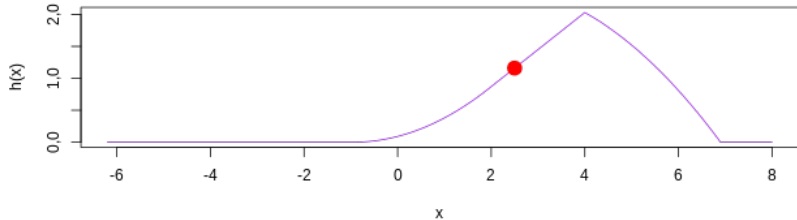
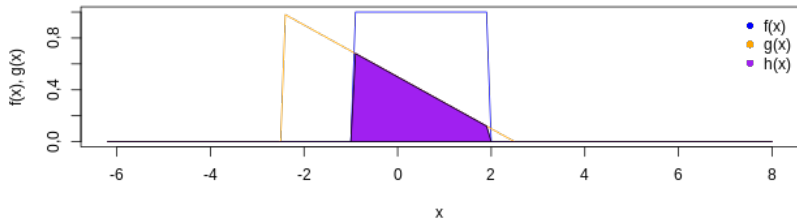
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



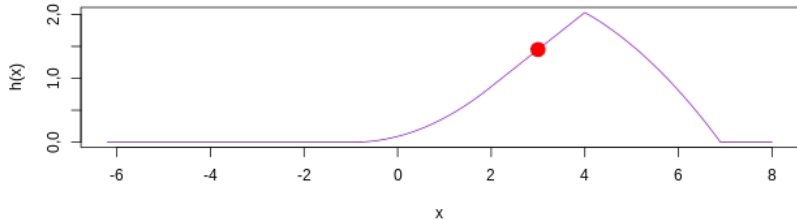
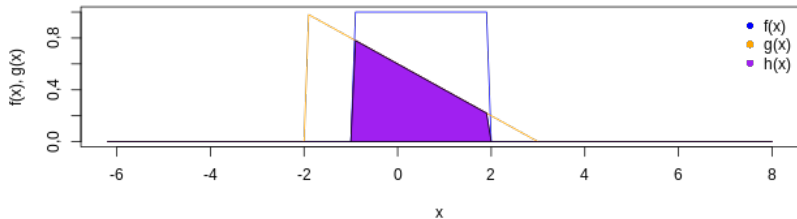
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



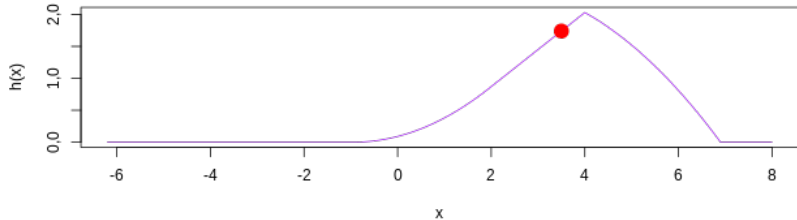
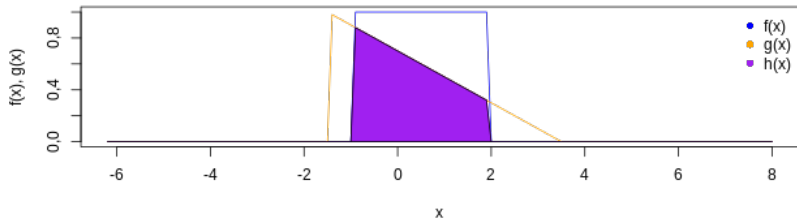
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



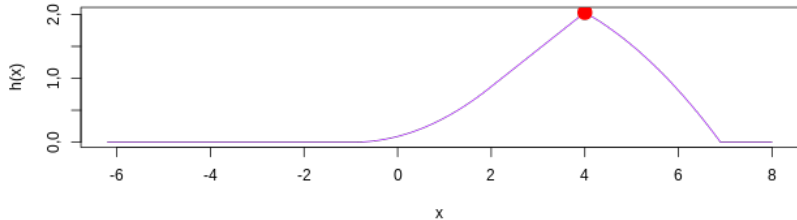
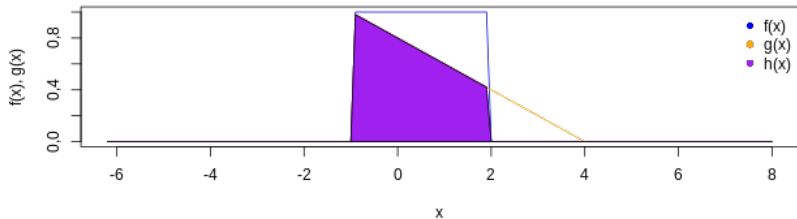
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



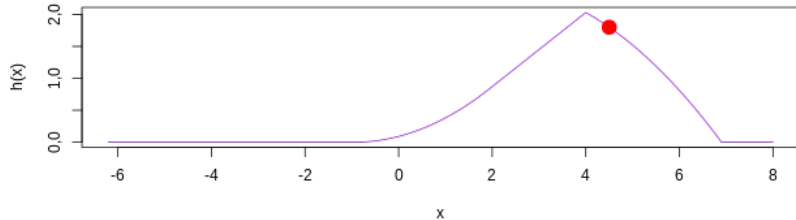
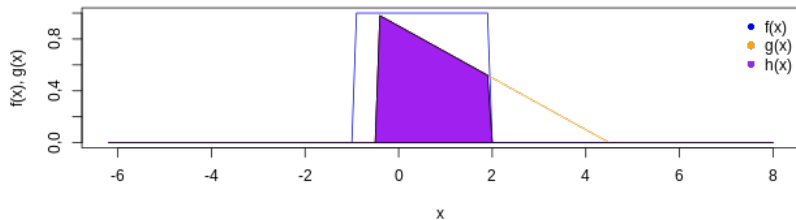
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



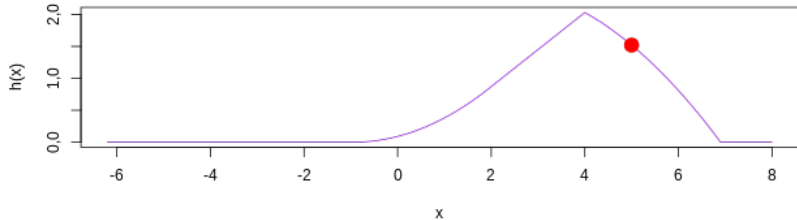
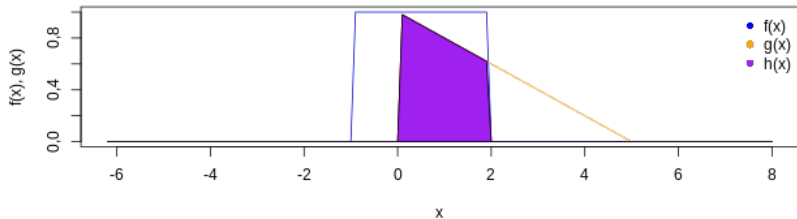
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



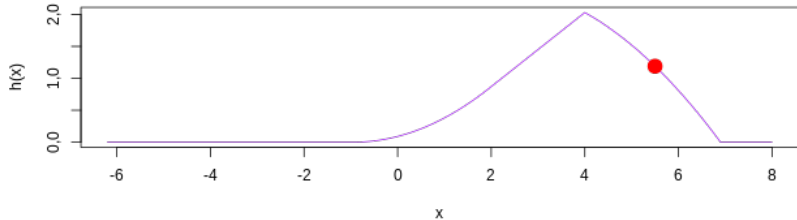
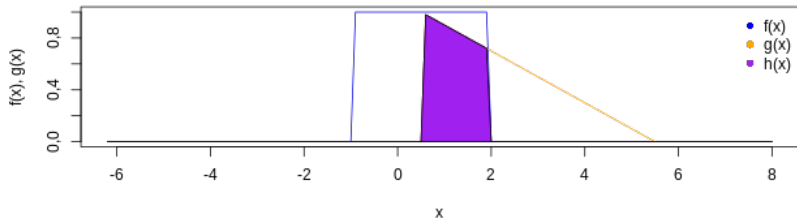
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



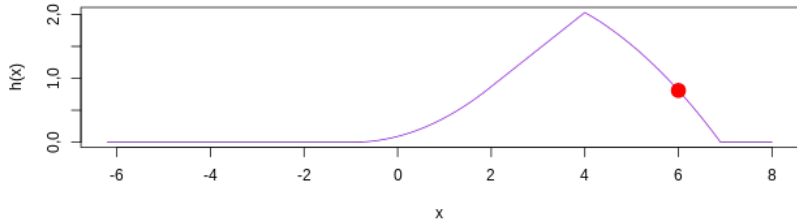
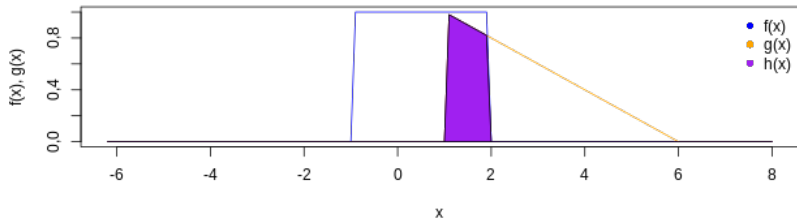
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



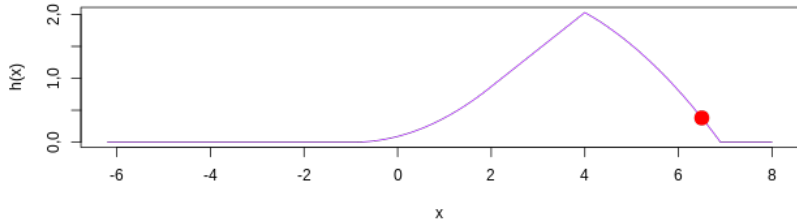
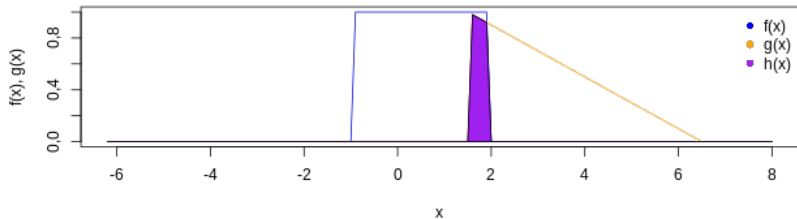
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



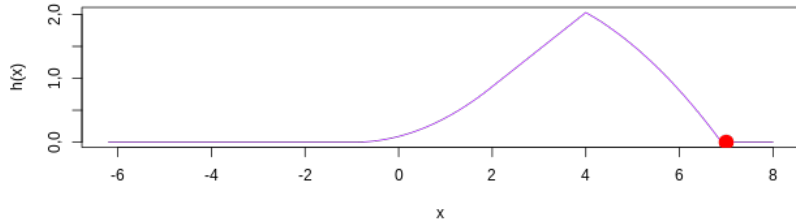
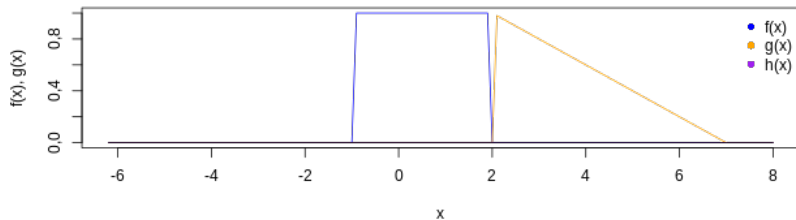
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



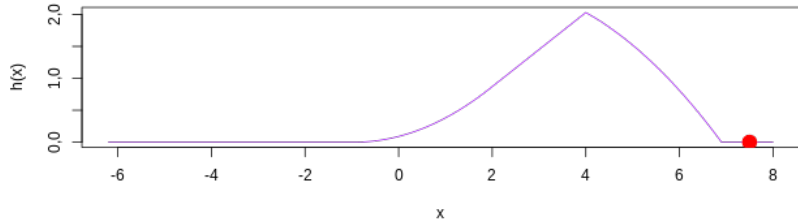
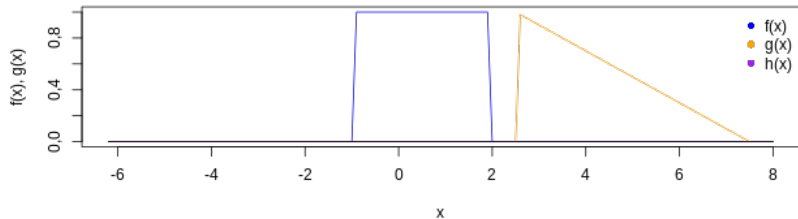
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



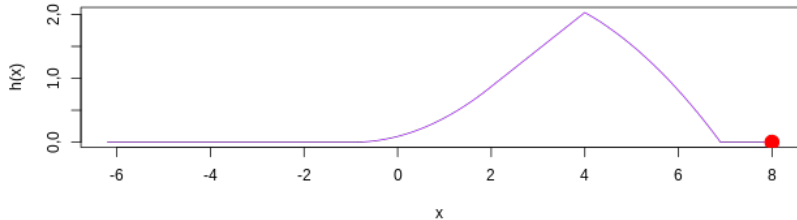
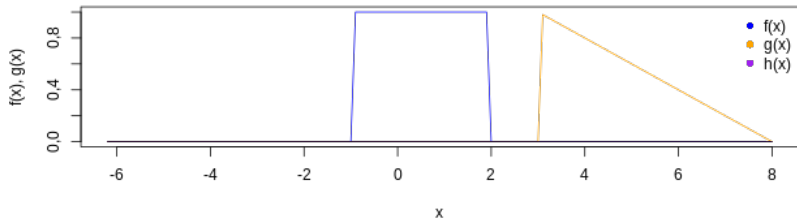
1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



1D CROSS CORRELATION ANIMATION

Cross-correlation of $f(x)$ with $g(x)$



CROSS CORRELATION VS. CONVOLUTION

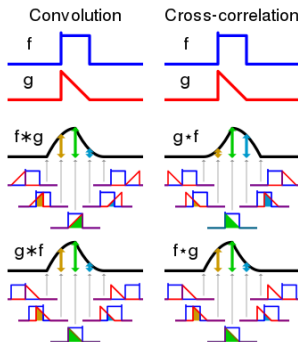


Figure: Comparison of convolution and cross-correlation

- Cross correlation is not commutative
- but often implemented instead of convolution in the practice.

Bonus 1: convolution in 2D

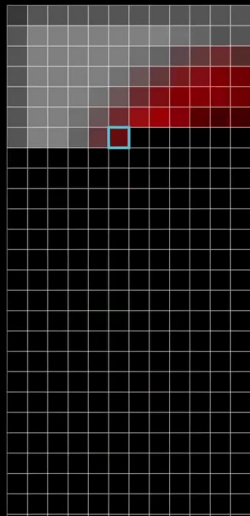
Every LMU page so far is one-dimensional. Here is the very same operation, blurring an image.

2D convolution: blurring an image

The moving average of the LMU pages, in two dimensions: slide a small **kernel** of weights over the image and average each neighborhood.

- ▶ A 3×3 **box kernel** - every weight $\frac{1}{9}$ - replaces each pixel with the mean of its 3×3 patch. The image blurs.
- ▶ Colors are vectors, so the average runs **per channel** (R, G, B).
- ▶ Pick different weights and you get sharpening or edge detection instead - a CNN *learns* those weights (L16).

The next two stills show the $\frac{1}{9}$ kernel marching across the image, then the weighted sum it computes at one stop.



$$\frac{1}{9} \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \end{bmatrix} + \frac{1}{9} \begin{bmatrix} 0.0 \\ 0.0 \\ 0.0 \end{bmatrix} + \frac{1}{9} \begin{bmatrix} 0.8 \\ 0.0 \\ 0.0 \end{bmatrix} + \frac{1}{9} \begin{bmatrix} 0.0 \\ 0.0 \\ 0.0 \end{bmatrix} + \frac{1}{9} \begin{bmatrix} 0.8 \\ 0.0 \\ 0.0 \end{bmatrix} + \frac{1}{9} \begin{bmatrix} 0.8 \\ 0.0 \\ 0.0 \end{bmatrix} + \frac{1}{9} \begin{bmatrix} 0.0 \\ 0.0 \\ 0.0 \end{bmatrix} + \frac{1}{9} \begin{bmatrix} 0.8 \\ 0.0 \\ 0.0 \end{bmatrix} + \frac{1}{9} \begin{bmatrix} 0.0 \\ 0.0 \\ 0.0 \end{bmatrix}$$



Bonus 2: computing convolutions fast

The convolution theorem is not just elegant - it hands you an algorithm three orders of magnitude faster than the naive slide-and-sum.

You have been convolving since school

Multiply two polynomials and watch the coefficients:

$$(1 + 2x + 3x^2)(4 + 5x + 6x^2) = 4 + 13x + 28x^2 + 27x^3 + 18x^4$$

Each output coefficient collects the pairwise products whose indices add up - the diagonals of the product table:

	1	2x	3x ²
4	4	8x	12x ²
5x	5x	10x ²	15x ³
6x ²	6x ²	12x ³	18x ⁴

Expanding a polynomial and collecting like terms *is* a convolution of the two coefficient lists: $(1, 2, 3) * (4, 5, 6) = (4, 13, 28, 27, 18)$.

Predict: how fast can we convolve?

Two lists of **100,000** random numbers. `numpy.convolve` averages **4.87 s**. **Predict** how long `scipy.signal.fftconvolve` - the same result, a cleverer algorithm - takes.

Predict: how fast can we convolve?

Two lists of **100,000** random numbers. `numpy.convolve` averages **4.87 s**. **Predict** how long `scipy.signal.fftconvolve` - the same result, a cleverer algorithm - takes.

4.3 ms - roughly **1000× faster**, identical output. Naive slide-and-sum fills an $n \times n$ table of pairwise products: $O(n^2)$. The fast route is $O(n \log n)$.

The trick: multiply in frequency

The convolution theorem (proved a few pages back) turns convolving into *multiplying* in the frequency domain:

$$f * g = \mathcal{F}^{-1}\{ \mathcal{F}\{f\} \cdot \mathcal{F}\{g\} \}.$$

- ▶ Treat each list as polynomial coefficients; **sample** both at the same points and the convolution becomes a cheap **pointwise** product.
- ▶ Sample at the **roots of unity** - evenly spaced on the unit circle. That choice is the **discrete Fourier transform**, and the **FFT** both evaluates it and inverts it in $O(n \log n)$.

FFT both lists $\rightarrow$ multiply pointwise $\rightarrow$ inverse FFT. That is what `fftconvolve` does - and why two n -digit numbers can be multiplied in $O(n \log n)$, not the $O(n^2)$ you learned in school.

Back to L16

- ▶ **Convolution** is a flip-and-slide dot product; it is commutative and has a clean identity in frequency (the convolution theorem).
- ▶ **CNNs compute cross-correlation** – the same operation without the flip – because the kernel is learned, so the flip is irrelevant.
- ▶ The frequency view is why a kernel is a **frequency filter**: box blur = low-pass, Sobel = high-pass (L16, section 2).
- ▶ It also makes convolution **fast**: FFT both lists, multiply pointwise, invert - $O(n \log n)$, the algorithm behind `fftconvolve`.

None of this changes how you build or train a CNN. It is the mathematics that justifies the one line L16 asked you to take on faith. Back to the main thread: **L17, the architectures**.